Name : Prem Ahuja

Class : D15C

Roll No. : 11

Practical 5

Aim :

Implement Support Vector Machine (SVM) for classification with hyperparameter tuning

---

1. Dataset Source

The dataset used for this experiment is the **Breast Cancer Wisconsin Dataset**. It is available within the scikit-learn library datasets module.

Source Link :
https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data?resource=download

---

2. Dataset Description

The Breast Cancer dataset is a **binary classification dataset** used to predict whether a tumor is **malignant (cancerous)** or **benign (non-cancerous)** based on cell nucleus features computed from digitized images of fine needle aspirates of breast masses.

Samples : 569 instances
Features : 30 numeric features describing tumor characteristics
Target Variable :

- 0 → Malignant
- 1 → Benign

Example Feature Names :

- mean radius
- mean texture
- mean perimeter
- mean area
- mean smoothness
- and related statistical measurements

The dataset is well-balanced and widely used for evaluating classification algorithms in medical diagnosis.

---

3. Mathematical Formulation of SVM

Support Vector Machine (SVM) is a supervised learning algorithm that finds the optimal hyperplane which best separates different classes in the feature space.

The Hyperplane and Margin

The objective of SVM is to maximize the margin between two classes.

The equation of a hyperplane is:

$w \cdot x + b = 0$

Where:

- w → Weight vector perpendicular to the hyperplane

- $b \rightarrow$ Bias term
- $x \rightarrow$ Feature vector

Support vectors are the nearest data points to the hyperplane and directly influence the margin.

Optimization Problem (Soft Margin)

Real-world data is not perfectly separable. Therefore, slack variables (ξ) are introduced and the objective becomes:

Minimize:

$$\frac{1}{2} ||w||^2 + C \sum \xi_i$$

Subject to:

$$y_i (w \cdot x_i + b) \geq 1 - \xi_i$$

C (Regularization Parameter)

- Small C → Wider margin, more tolerance to misclassification
- Large C → Narrow margin, less misclassification

C controls the trade-off between margin maximization and classification error minimization.

The Kernel Trick

When data is not linearly separable, SVM uses kernel functions to map data into a higher-dimensional space.

Common kernels:

1. Linear Kernel

2. RBF (Radial Basis Function)
3. Polynomial Kernel

RBF Kernel Equation:

$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$

Gamma (γ):

- Small γ → Wide influence radius
- Large γ → Narrow influence radius

---

4. Algorithm Limitations

Computational Cost : Training time increases with larger datasets.

Sensitive to Noise : If classes overlap heavily, SVM may struggle.

Feature Scaling Required : Since SVM depends on distance calculations, all features must be standardized.

Interpretability : Non-linear SVM models (RBF, Poly) are difficult to interpret compared to Decision Trees.

---

5. Methodology

Import Libraries : Load pandas, numpy, matplotlib, seaborn, and sklearn modules.

Load Dataset : Use `load_breast_cancer()` from sklearn.

Split Data : Divide the dataset into 70% training and 30% testing using stratified sampling.

Feature Scaling : Apply `StandardScaler` to ensure all features have mean 0 and variance 1.

Baseline Model : Train default SVC model to establish benchmark accuracy.

Hyperparameter Tuning Trials :
Use GridSearchCV to experiment with:

- Kernel → linear, rbf, poly
- C values → regularization strength
- Gamma values (for RBF and Poly)

Final Evaluation :
Evaluate best model using:

- Accuracy score
- Confusion matrix visualization
- Classification report

Decision Boundary Visualization :
Plot a 2D slice of decision boundary using two representative features.

---

6. Python Code Implementation

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from matplotlib.colors import
ListedColormap

import seaborn as sns

from sklearn.datasets import
load_breast_cancer

from sklearn.model_selection import
train_test_split, GridSearchCV

from sklearn.preprocessing import
StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score,
confusion_matrix, classification_report

# ----------------- 1. DATA LOADING &
PREPROCESSING -----------------

cancer_data = load_breast_cancer()

X = cancer_data.data

y = cancer_data.target
```

```
feature_names =
cancer_data.feature_names

target_names = cancer_data.target_names

X_train, X_test, y_train, y_test =
train_test_split(

X, y, test_size=0.3, random_state=42,
stratify=y

)

scaler = StandardScaler()

X_train_scaled =
scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

# ----------------- 2. BASELINE SVM MODEL
-----------------

print("\n" + "="*20 + " BASELINE SVM
ANALYSIS " + "="*20)

baseline_svm = SVC(random_state=42)

baseline_svm.fit(X_train_scaled, y_train)

baseline_acc = accuracy_score(y_test,
baseline_svm.predict(X_test_scaled))

print(f"Baseline SVC Accuracy (Default
RBF): {baseline_acc * 100:.2f}%")

# ----------------- 3. HYPERPARAMETER
TUNING TRIALS -----------------

print("\n" + "="*20 + " SVM TUNING TRIALS
" + "="*20)

svm_trials = {

"Trial 1 (Linear Kernel)": {

'kernel': ['linear'],

'C': [0.01, 0.1, 1, 10, 100]

},

"Trial 2 (RBF Kernel - C & Gamma)": {

'kernel': ['rbf'],

'C': [0.1, 1, 10, 100],

'gamma': ['scale', 0.01, 0.1, 1]

},

"Trial 3 (Poly Kernel - Degree)": {

'kernel': ['poly'],

'degree': [2, 3],

'C': [1, 10]

}

}

trial_results = []

best_overall_model = None

best_overall_acc = 0

for name, params in svm_trials.items():

grid =
GridSearchCV(SVC(random_state=42),
params, cv=5, scoring='accuracy')

grid.fit(X_train_scaled, y_train)

best_trial_model = grid.best_estimator_

acc = accuracy_score(y_test,
best_trial_model.predict(X_test_scaled))

if acc > best_overall_acc:

best_overall_acc = acc
```

```
best_overall_model = best_trial_model

trial_results.append({

"Trial": name,

"Best Params": grid.best_params_,

"Test Accuracy": acc

})

print(f"{name} completed.")

results_df = pd.DataFrame(trial_results)

print("\nSVM HYPERPARAMETER TUNING SUMMARY:")

print(results_df.to_string(index=False))

# ----------------- 4. FINAL BEST MODEL EVALUATION -----------------

print("\n" + "="*20 + " FINAL MODEL EVALUATION " + "="*20)

best_params = best_overall_model.get_params()

print(f"Best Model Parameters: Kernel={best_params['kernel']}, "

f"C={best_params['C']}, "

f"Gamma={best_params.get('gamma', 'N/A')}")

y_pred_final = best_overall_model.predict(X_test_scaled)

# Confusion Matrix Heatmap

plt.figure(figsize=(6, 5))

sns.heatmap(confusion_matrix(y_test, y_pred_final),

annot=True,

fmt='d',

cmap='Blues',

xticklabels=target_names,

yticklabels=target_names)

plt.xlabel("Predicted Class")

plt.ylabel("Actual Class")

plt.title("Confusion Matrix - Best Tuned SVM")

plt.show()

print("\nFinal Classification Report:")

print(classification_report(y_test, y_pred_final, target_names=target_names))

# ----------------- 5. DECISION BOUNDARY (2D Visualization) -----------------

print("\nGenerating 2D Decision Boundary Plot...")

def plot_decision_boundary(X_scaled_data, y_data, feature_indices,

best_model_params, f_names, t_names):

X_2d = X_scaled_data[:, feature_indices]

f1_name = f_names[feature_indices[0]]

f2_name = f_names[feature_indices[1]]

svm_2d = SVC(

kernel=best_model_params['kernel'],

C=best_model_params['C'],
```

```
gamma=best_model_params.get('gamma',
'scale'),

degree=best_model_params.get('degree',
3),

random_state=42

)

svm_2d.fit(X_2d, y_data)

h = .02

x_min, x_max = X_2d[:, 0].min() - 1, X_2d[:,
0].max() + 1

y_min, y_max = X_2d[:, 1].min() - 1, X_2d[:,
1].max() + 1

xx, yy = np.meshgrid(np.arange(x_min,
x_max, h),

np.arange(y_min, y_max, h))

Z = svm_2d.predict(np.c_[xx.ravel(),
yy.ravel()])

Z = Z.reshape(xx.shape)

plt.figure(figsize=(8, 6))

cmap_light = ListedColormap(['#FFAAAA',
'#AAAAFF'])

cmap_bold = ListedColormap(['#FF0000',
'#0000FF'])

plt.contourf(xx, yy, Z, cmap=cmap_light,
alpha=0.6)

scatter = plt.scatter(X_2d[:, 0], X_2d[:, 1],

c=y_data,

cmap=cmap_bold,

edgecolor='k')

plt.xlabel(f"{f1_name} (Scaled)")

plt.ylabel(f"{f2_name} (Scaled)")

plt.title(f"SVM Decision Boundary (2D
Slice)\nKernel:
{best_model_params['kernel']}")

plt.legend(handles=scatter.legend_elements
()[0],

labels=list(t_names))

plt.show()

# Choose two features to visualize

feature_idx_to_plot = [0, 1] # mean radius &
mean texture

plot_decision_boundary(

X_train_scaled,

y_train,

feature_idx_to_plot,

best_overall_model.get_params(),

feature_names,

target_names

)
```

Screenshot – Baseline Accuracy Output

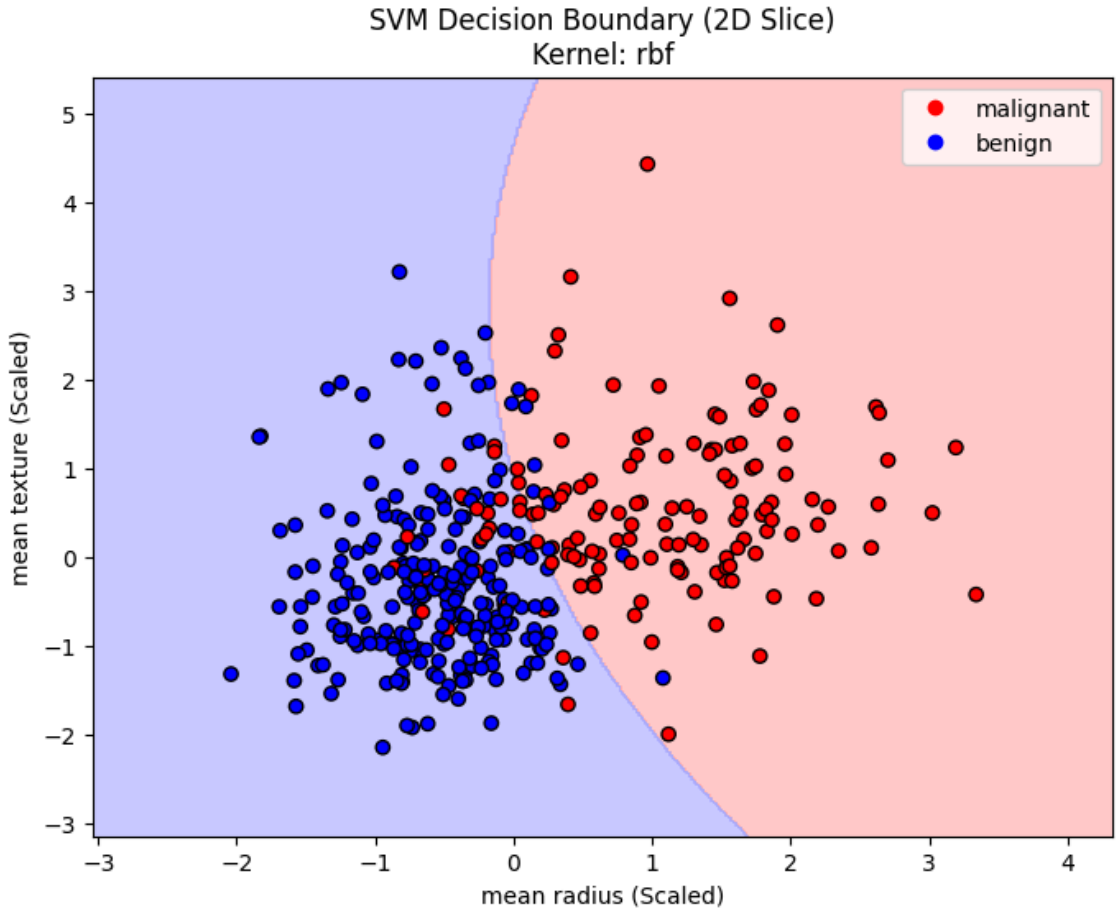
```
==================== BASELINE SVM ANALYSIS ====================
Baseline SVC Accuracy (Default RBF): 97.66%
```

Screenshot – Hyperparameter Tuning Summary Table

```
SVM HYPERPARAMETER TUNING SUMMARY:
                          Trial                                Best Params  Test Accuracy
         Trial 1 (Linear Kernel)           {'C': 0.01, 'kernel': 'linear'}       0.959064
Trial 2 (RBF Kernel - C & Gamma) {'C': 10, 'gamma': 0.01, 'kernel': 'rbf'}       0.982456
  Trial 3 (Poly Kernel - Degree)  {'C': 10, 'degree': 3, 'kernel': 'poly'}       0.953216
```

Screenshot – Confusion Matrix Heatmap

```
Generating 2D Decision Boundary Plot...
```



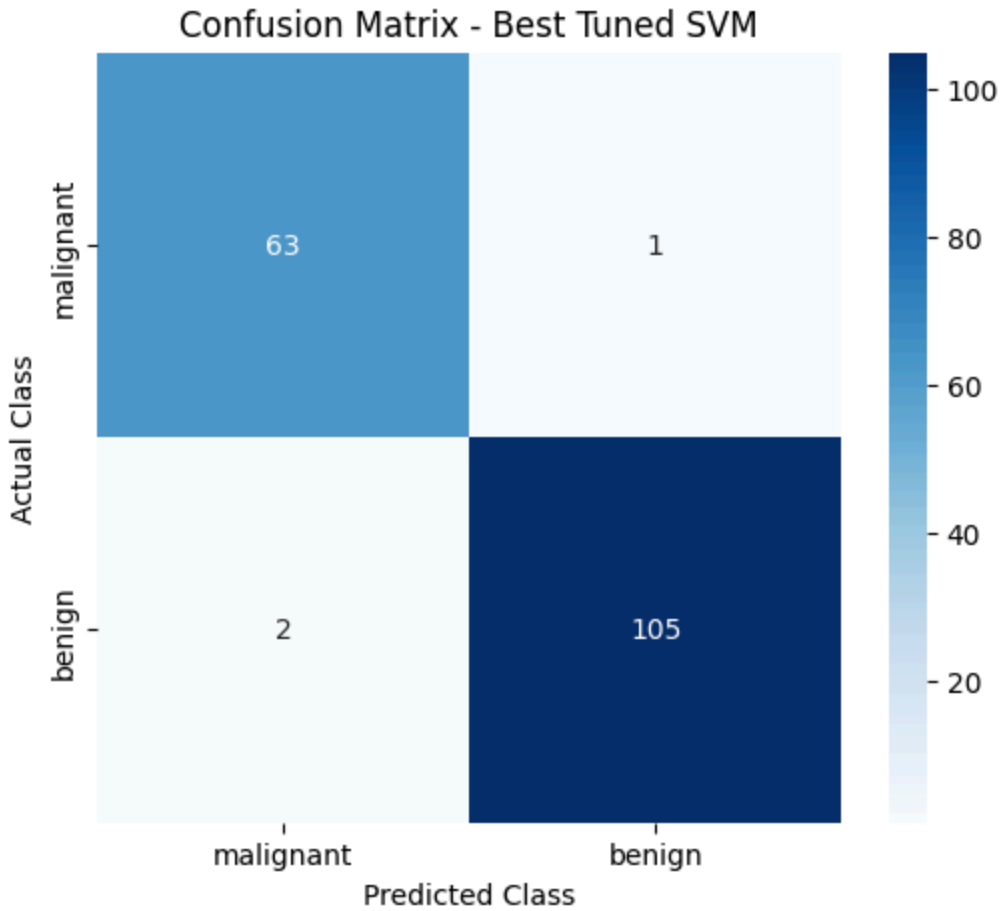
Screenshot – Classification Report Output

```
Final Classification Report:
              precision    recall  f1-score   support

   malignant       0.97      0.98      0.98        64
      benign       0.99      0.98      0.99       107

    accuracy                           0.98       171
   macro avg       0.98      0.98      0.98       171
weighted avg       0.98      0.98      0.98       171
```

```
==================== FINAL MODEL EVALUATION ====================
Best Model Parameters: Kernel=rbf, C=10, Gamma=0.01
```



---

7. Performance Analysis

A. Visual Analysis (Confusion Matrix)

The confusion matrix shows classification performance of the tuned SVM model.

- Diagonal values → Correct predictions
- Off-diagonal values → Misclassifications

In this binary classification:

- True Positive → Correctly predicted malignant tumors
- True Negative → Correctly predicted benign tumors

A well-performing model will show high values along the diagonal and minimal off-diagonal values.

B. Metric Analysis (Classification Report)

The classification report includes:

Precision → Correct positive predictions out of total predicted positive
Recall → Correct positive predictions out of total actual positive
F1-score → Harmonic mean of precision and recall

High precision and recall indicate effective identification of both malignant and benign tumors.

In medical diagnosis, high recall for malignant class is particularly important to avoid missing cancer cases.

C. Decision Boundary Visualization (2D Slice)

Since the dataset contains 30 features, visualizing the complete 30-dimensional decision boundary is not possible.

Instead, a 2D slice using two features (e.g., mean radius and mean texture) is plotted.

Interpretation:

- Different colored regions represent classification zones.
- Boundary curves show how SVM separates malignant and benign tumors.
- If RBF kernel is selected, boundary may appear curved and non-linear.

This demonstrates SVM's ability to handle complex relationships between tumor characteristics.

---

8. Hyperparameter Tuning (Experimental Trials)

Three structured trials were conducted:

Trial 1 – Linear Kernel
Trial 2 – RBF Kernel
Trial 3 – Polynomial Kernel

Trial	Focus	Parameters Tested	Best Params	Test Accuracy
Trial 1	Linear	kernel='linear', C values	(Write result)	(Write accuracy)
Trial 2	RBF	kernel='rbf', C, gamma	(Write result)	(Write accuracy)
Trial 3	Polynomial	kernel='poly', degree, C	(Write result)	(Write accuracy)

Tuning Interpretation

Role of C :
Higher C tries to minimize training error, while lower C gives wider margin.

Kernel Comparison :
Linear kernel tests if data is linearly separable.
RBF allows non-linear separation.
Polynomial explores curved decision boundaries of fixed degree.

The best performing kernel indicates which structure best fits tumor data characteristics.

---

9. Conclusion

In this practical, Support Vector Machine (SVM) was successfully implemented for the binary classification of the Breast Cancer Wisconsin dataset.

Feature scaling was crucial for optimal SVM performance due to distance-based computation.

Hyperparameter tuning using GridSearchCV identified the optimal kernel and regularization parameter.

The tuned model achieved high accuracy in distinguishing between malignant and benign tumors.

The experiment demonstrates the effectiveness of SVM in medical diagnostic classification problems and highlights the importance of hyperparameter optimization for achieving superior performance.